Technical Design Document (TDD)

COMP710 Game Programming – Group Project. A 2D tower defense game with a mining / underground theme. Players draw their own path, build towers and survive endless scaling waves.

1. Target Platform, Development Environment & Tools

- **Target Platform:** Windows PC (64-bit)
- **Development Environment:** Visual Studio 2022, C++17
- **Build System:** Visual Studio Solution (Pathfinder.sln) + MSBuild
- **Version Control:** GitHub (team repository, individual branches + Pull Requests)
- **Debugging Tools:** Visual Studio Debugger, Dear ImGui debug panel, custom LogManager output
- **Profiling:** Built-in frame timer, FPS display via ImGui

2. Required Technologies & Libraries

Library / Tech	Purpose in the project
SDL2 2.0.22	Window creation, input events, high-resolution timing.
SDL2_image 2.6.1	Loading PNG sprite sheets and textures.
SDL2_ttf 2.22.0	Font rendering for the HUD via DynamicText (gold, lives, wave, instructions).
OpenGL + GLEW 2.1.0	Hardware-accelerated 2D drawing (Renderer, Shader, VertexArray, Matrix4).
Box2D	Physics world. Tower attack radius is a circle sensor; enemies and projectiles carry collision shapes.
FMOD	Background music and sound effects (wave start, game over).
Dear ImGui	Debug overlay and runtime value tuning.
AUT COMP710 Framework	Provided base: Game loop, Scene, Sprite, AnimatedSprite, Texture/TextureManager, InputSystem, LogManager.

3. Key Technical Challenges & Solutions

Challenge	Solution
A tower must know which enemies are inside its attack radius every frame.	Each Tower owns a Box2D circle sensor (radius = TOWER_RADIUS_TILES_DEFAULT × tileSize). Box2D reports the overlapping enemy shapes into EnemyInRadius, so we avoid a brute-force O(n²) distance check across all enemies.
Enemies must follow the path the player drew, not a fixed track.	The path is stored as a doubly-linked list of Tile objects (NextPosition / PrevPosition). At runtime each enemy just walks toward its NextPosition tile, so no pathfinding search is needed during combat.
The “tunnel problem”: a player could draw a very long path to get maximum tower coverage.	Handled by the economy instead of a hard rule. Kill bounty decays each wave (BOUNTY_DECAY) and only pays inside a time window (PAY_WINDOW); a one-off short-path bonus rewards paths shorter than PATH_PAR tiles. Enemy speed can also be scaled by path length.
Adding new towers /	Content is data-driven. TowerData, EnemyData and

Challenge	Solution
enemies / projectiles without rewriting logic.	ProjectileData rows are read from an .ini file by IniParser into the GameData singleton maps. Adding content = new data row + sprite, no logic recompile.
Projectiles need both homing and piercing behaviour.	Projectile supports either a homing target (b2ShapeId) or piercing through up to maxenemies. They are kept mutually exclusive on purpose to avoid edge cases.
Economy balancing values scattered through the code make tuning painful.	Every gold / economy number lives in one file, EconomyConfig.h, so re-balancing is a single-file change.
Spawning hit / explosion effects every frame would allocate constantly.	A fixed-size particle pool (PARTICLE_POOL_SIZE) is reused for bursts instead of allocating new particles.

4. Architecture Overview (with UML)

4.1 Core Module Division

- **SceneGame:** the main gameplay scene. Owns the tile grid, the towers / enemies / projectiles lists and the path maker; runs the update / render / input loop; tracks wave, lives and gold; handles Game Over and restart.
- **TileList & Tile:** the 15×15 grid. Each Tile stores its position and flags (isPath, isOccupied). TileList builds and holds the path the player draws.
- **Pathmaker:** turns the player's selected tiles into the ordered path the enemies follow.
- **Tower:** placed on a tile; has range, fire rate and damage; scans the enemies in its radius and creates Projectiles.
- **Projectile:** moves toward / through enemies, applies damage and effects (slow, damage-over-time) and checks collision.
- **Enemy:** spawned per wave; moves along the path; health scales with the wave number; takes damage, dies, or leaks to the end and costs a life.
- **UIShop:** tower selection and purchase UI (shop slots + a side panel to inspect / sell a placed tower).
- **ParticleSystem:** explosion and hit feedback effects.
- **GameData (singleton):** central game state and configuration – gold, lives, wave – plus the data tables for towers, enemies and projectiles.

4.2 UML Class Diagram

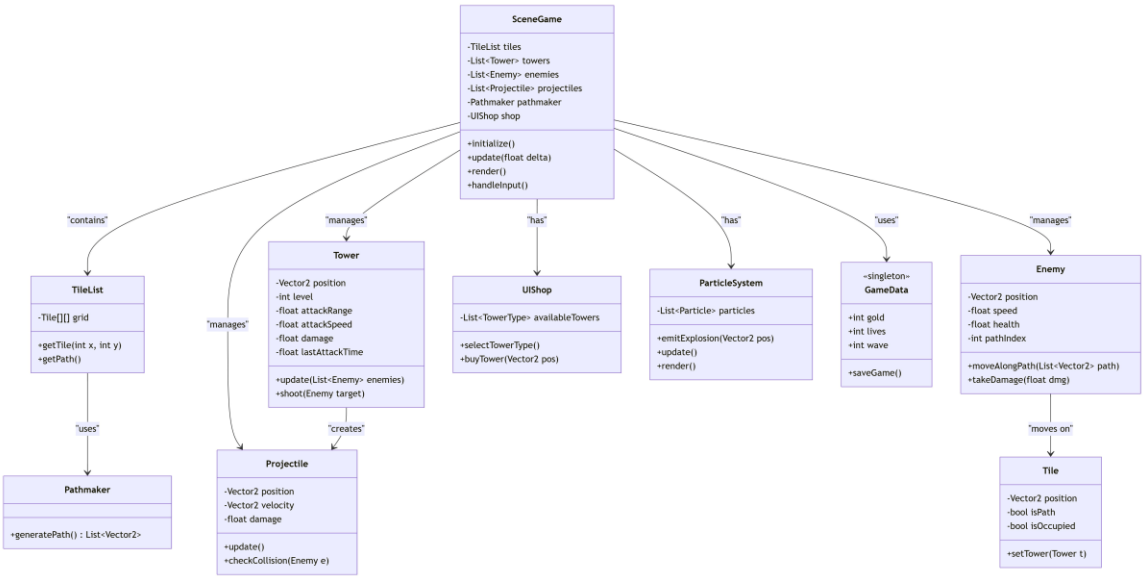

Figure 1. Pathseeker class diagram – SceneGame composes the grid, path, towers, enemies, projectiles, shop and particle system; GameData is a shared singleton.

4.3 State Machine Diagram – Game Flow

The game runs through a small set of states. There is no break between waves once combat starts; every 10th wave spawns a boss / elite enemy.

State	What happens	Transition out
Splash	Title / splash screen (SceneSplash).	Any key → Path Build
Path Build	Player draws a path of tiles from Start to End on the 15×15 grid.	Valid path finished → Combat
Combat	Waves spawn continuously, enemy HP scales with wave, player places / upgrades / sells towers and earns gold.	Lives = 0 → Game Over
Game Over	Game Over sprite + restart prompt shown, gameplay frozen.	Press R → Restart
Restart	Reset gold, lives, wave, clear towers / enemies / projectiles.	→ Path Build

4.4 Sequence Diagram – Tower Attacks an Enemy

Per-frame interaction when a tower acquires and fires at an enemy:

- SceneGame calls Tower::Process(deltaTime) on every tower.
- The tower's Box2D radius sensor reports the enemy shapes currently overlapping it (EnemyInRadius).
- If the fire timer is ready and at least one enemy is in range, the tower picks a target and creates a Projectile (passing damage, pierce, homing and speed from its data).
- SceneGame calls Projectile::Process(deltaTime); the projectile moves toward / through the target.
- On collision the projectile calls Enemy::TakeDamage(amount) and any TakeEffect(amount) (slow / poison).
- If Enemy::IsDead(), SceneGame removes the enemy, spawns a particle burst and awards kill bounty via AddGold(); if the enemy reached the end instead, the player loses a life.

5. Technical Risks & Mitigation

Risk	Mitigation
Long-path economy exploit (player farms coverage with an over-long path).	Bounty decay + pay window + short-path bonus; optional enemy speed scaling by path length.
Dangling Box2D references (a projectile's target enemy dies before impact).	Validate the target body / shape is still alive before using it; clear dead targets.
Weak team communication / unclear task split.	Data-driven, modular design lets members work on separate towers / enemies independently; GitHub branches + PR to integrate.
Scope creep (6 towers × upgrades, relics, bosses).	MoSCoW prioritisation – upgrades, relics and bosses are Should / Could, not Must.
Performance drop with many enemies + projectiles on screen.	Object pooling for particles, Box2D for collision, capped projectile lifetime, removal of dead entities each frame.
Asset / sprite-sheet inconsistency.	Shared tile size; AnimatedSprite + consistent sprite sheets; assets loaded through TextureManager.

6. Feature Scope (MoSCoW)

Must have:

- 15×15 tile grid with a player-drawn path (Start → End).
- Wave spawning with enemy HP scaling by wave number; no break between waves.
- At least the basic towers (e.g. Torchman) that detect enemies in radius and fire projectiles.
- Projectiles that deal damage to enemies.
- Gold economy: starting gold, kill bounty, spending gold to place towers.
- Lives, Game Over state and restart (R key).
- HUD showing gold, lives and wave; splash screen.

Should have:

- Multiple tower types (Torchman, Snowman, Spear Thrower, Pickaxe Thrower, Pulse Bot, Demoman).
- Sell a placed tower from the side panel.
- Status effects: slow (Snowman) and damage-over-time / poison.
- Short-path bonus and per-wave bounty decay.
- FMOD background music and sound effects; particle hit effects; instructions overlay.

Could have:

- Tower upgrades (choose 2 of 3 upgrade options).
- Boss / elite enemy every 10th wave with a gimmick (Evil Snowman, The Blob).
- Relics dropped by bosses that synergise with specific towers.
- Per-tower custom attack radius; Pickaxe "Gold Striker" bonus gold on kill.

Won't have:

- Networking / multiplayer.
- 3D graphics.
- Full save / load persistence across sessions.
- Story / campaign mode.

Sign Off:
Jesse Wang
Cody Bennet
Martin Yan